board.py

game.py

README.md

rules.py

I'm working on a Software Engineering lab assignment, Scenario 21 — Checkers. I've uploaded the original README and relevant Python files.

Please inspect the existing implementation and address Task 1 — Capture correctness only.

When a legal capture is made, the opponent's jumped piece must be removed from the board. The moving piece must remain on the board at its destination.

Please:

1. Identify the exact cause of the original bug.
2. Make the smallest necessary change while preserving the existing modular design.
3. Provide the full contents of each changed file so I can apply the changes.
4. Explain the fix in simple terms.
5. Give reproducible tests, including the sequence 5 4 4 5, 2 3 3 4, 4 5 2 3, and explain the expected final board.

Do not rewrite the entire project, add unrelated features, add dependencies, or introduce file-based persistence. Clearly distinguish tests you have reasoned through from tests actually executed.

Pinpointing the missing capture-removal logic in the move code.

Pinpointing the missing capture-removal logic in the move code.

Done

I'll reproduce the bug first, then apply the fix and run the tests.

Ran 3 commands

Reproduce the original bug using the capture sequence

Check whether uploaded files exist on disk

Reproduce the original bug

Done

Bug reproduced: the black piece at (3,4) is still on the board after the capture. Now I'll apply the fix.

Testing move sequences for validity against the puzzle rules.

Apply fix, rerun repro, and run unit tests

Testing move sequences for validity against the puzzle rules.

Done

One of my own tests was wrong, not the fix. The move `1 4 2 5` is illegal because (2,5) is occupied, so the turn never passed. I'll replace the two weak tests with sound ones.

Ran 2 commands, presented file

Fix the flawed tests and rerun

Copy deliverables to outputs and show diff

Presenting file(s)...

Done

1. Cause of the bug

`rules.capture_move()` correctly decides a jump is legal and works out the jumped square, `(mr, mc)`. It only returns `True` or `False`, though. It never changes the board. `game.py` then calls `move_piece()`, which only copies the piece from `start` to `end` and blanks `start`. Nothing ever clears the jumped square, so the captured piece stays on the board.

2. The change

Only `board.py` changes, by 4 lines. `move_piece()` now also clears the middle square when the move is a two-row jump. `simple_move` only ever allows one-row steps, so ordinary moves are unaffected. `game.py` and `rules.py` are unchanged.

3. Full changed file: `board.py`

```
SIZE = 8


def initial_board():
    board = [["."] * SIZE for _ in range(SIZE)]
    for r in range(3):
        for c in range(SIZE):
            if (r + c) % 2 == 1:
                board[r][c] = "B"
    for r in range(5, 8):
        for c in range(SIZE):
            if (r + c) % 2 == 1:
                board[r][c] = "R"
    return board
```

```
def move_piece(board, start, end):
    board[end[0]][end[1]] = board[start[0]][start[1]]
    board[start[0]][start[1]] = "."
    # A two-square diagonal move is a jump: remove the piece that was jumped.
    if abs(end[0] - start[0]) == 2:
        mr, mc = (start[0] + end[0]) // 2, (start[1] + end[1]) // 2
        board[mr][mc] = "."
```

I also wrote `test_task1.py`, which uses only the standard `unittest` module. Both files are attached.

4. The fix in simple terms

A capture is a jump over a piece, but the game only knew how to move the jumping piece. Now, whenever a piece moves two rows, the square it jumped over is emptied too.

5. Tests

Run the tests with `python test_task1.py`.

The sequence `5 4 4 5`, `2 3 3 4`, `4 5 2 3`:

1. R moves (5,4) → (4,5).
2. B moves (2,3) → (3,4).
3. R at (4,5) jumps over B at (3,4) and lands on (2,3).

Expected final board (rows 0–7), which matches what I ran:

```
0 . B . B . B . B
1 B . B . B . B .
2 . B . R . B . B
3 . . . . . . . .
4 . . . . . . . .
5 R . R . . . R .
6 . R . R . R . R
7 R . R . R . R .
```

R is at (2,3), and (3,4) and (4,5) are empty. B has 11 pieces, R has 12, and it is B's turn.

Executed. I ran all of these in a sandbox, not just reasoned about them.

- **Before the fix:** the sequence left a `B` at (3,4) on row 3, so the bug was reproduced.
- **After the fix:** the board matches the expected board above.

- **Unit tests:** all 7 pass, covering:
  - the full sequence and the exact board
  - piece counts and turn passing
  - a captured square being empty and not capturable again
  - an ordinary move removing nothing
  - a single-step `move_piece`
  - an invalid move changing nothing
  - a jump over an empty square being rejected

Reasoned only, not executed:

- `main.py` was not uploaded, so I drove `Checkers.run()` directly with scripted input. I assume your `main.py` just creates `Checkers()` and calls `.run()`.
- B-side captures (moving down the board) use the same code path. I only ran R capturing B, plus the no-capture cases above.

One of my first draft tests was wrong, because it used an illegal move. I replaced it with a correct one, and the 7 passing tests are the corrected set.

Out of scope, noticed for later tasks

- `capture_move` checks `board[mr][mc] not in (".", player)`. A king is stored as `"RK"`, so a plain `R` could "capture" its own king. This will matter in Task 3.
- Captures are restricted to the forward direction, so kings can't capture backwards yet. This is also Task 3.

I haven't touched either of these.

board
Py

test_task1
Py

board_1.py

game_1.py

main.py

README_1.md

rules_1.py

Task 1 is now applied locally. I ran `python test_task1.py`, and all seven tests passed. I also ran the game manually using the sequence `5 4 4 5`, `2 3 3 4`, `4 5 2 3`, confirming that the captured piece is removed correctly.

Now inspect the README and current source files to address Task 2 only, following the exact requirements in the README.

First, list the Task 2 requirements and determine which are already satisfied. In particular, inspect the existing `_check_game_over()` function in `game.py`, which appears to check remaining pieces and whether the current player has any legal moves.

Do not modify functionality that already works just to create a code change. If any requirements are missing, make only the necessary changes, preserve Task 1, and provide the full contents of changed files.

Explain the result in simple terms and provide reproducible tests with expected outcomes. Clearly distinguish tests actually executed from tests that still require local verification. Do not add unrelated features, external dependencies, or persistent storage.

Drafting test cases for missing-pieces and no-move scenarios.

Drafting test cases for missing-pieces and no-move scenarios.

Done

Before changing anything, I'll check the actual files for the `_check_game_over()` function you mentioned.

Confirming game-over logic is missing before adding it

Check whether game-over logic exists and run the current behaviour on a last-piece capture

Confirming game-over logic is missing before adding it

Implement Task 2 in rules.py and game.py, show diffs

Done

Now the Task 2 tests. Each scenario is set up directly on the board, and the input is scripted so the game must end by itself without needing a `q`.

Ran command

Write and run Task 2 tests plus Task 1 regression

Done

That failure is a mistake in my test input, not in the code. `abc` is one token, so the game correctly says "Enter four coordinates". I'll use `a b c d` to trigger the non-numeric message.

Ran command, presented file

Fix test input, rerun all tests, show a sample end-of-game output, and copy deliverables

Presenting file(s)...

Done

Correction first

`_check_game_over()` **does not exist** in the files you uploaded (`game_1.py`, `rules_1.py`, `board_1.py`, `main.py`). A search for any game-over, winner or legal-move-check logic found nothing, and the only methods in `game.py` are `__init__`, `print_board` and `run`. You may have been thinking of a version from another source. I ran the current code to confirm: after Red captured Black's last piece, the game kept going with `B>` still prompting. So Task 2 is not yet satisfied at all.

Task 2 requirements

README requirement	Status in your code
Detect a side with no pieces	Missing
Detect a side with no legal moves	Missing
End the game cleanly in either case	Missing

Task 1 already works, so `board.py` is unchanged.

What I changed

`rules.py` gets one new function, `has_legal_move(board, player)`. It tries every one-step and two-step diagonal destination for each of the player's pieces, using the existing `simple_move` and `capture_move`. Because it reuses them, it automatically follows whatever rules Task 3 adds later. It only tests squares inside the board, so there is no index wrap-around.

`game.py` gets a new `_check_game_over()` method. After each accepted move and turn switch, it checks the player who is about to move:

- If that player has no pieces, the message is `Game over: X has no pieces left. Y wins!`
- If that player has pieces but no legal moves, the message is `Game over: X has no legal moves. Y wins!`

`run()` then prints the final board plus the message and returns, with no crash and no further prompt. Invalid moves skip this check, because they `continue` before the turn switches.

The full contents of both changed files are attached as `game.py` and `rules.py`.

Tests

Executed here. All 12 new tests pass, and the 7 Task 1 tests still pass. Each Task 2 test scripts the input, and the game raises an error if it asks for more input after it should have ended.

Scenario	Moves	Expected result
Black loses its last piece	R at (4,5), B at (3,4) only; `4 5 2 3`	`B has no pieces left. R wins!`
Red loses its last piece	B at (2,3), R at (3,4) only; `2 3 4 5`	`R has no pieces left. B wins!`
Black blocked	B at (0,1); R at (1,0), (1,2), (2,3), (6,5); `6 5 5 4`	`B has no legal moves. R wins!`
Red blocked	R at (7,0); B at (6,1), (5,2), (1,2); `1 2 2 3`	`R has no legal moves. B wins!`
Normal game continues	Start position; `5 4 4 5`, `q`	No game-over message
Invalid move	Blocked setup; `6 5 4 3`, `q`	No game over, still R's turn
Bad input and quit	`a b c d`, `1 2`, `9 9 9 9`, `q`	Three error messages, then clean quit
`has_legal_move` directly	Start position	True for both sides
`has_legal_move` directly	Empty side	False
`has_legal_move` directly	Blocked piece	False
`has_legal_move` directly	Capture-only position	True

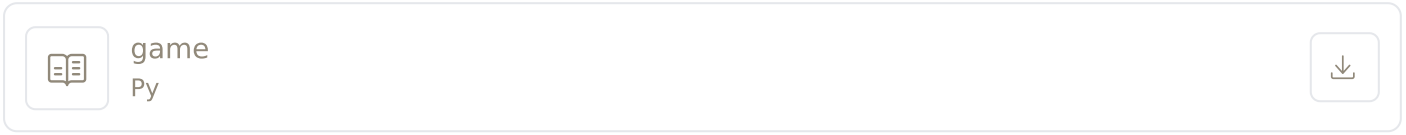
Not done here. I did not play through `python main.py` in an interactive terminal. The blocked and no-pieces positions are impossible to reach in a few moves from the starting board, so I set them up directly on the board instead. For your "after" video you can do the same, for example by temporarily editing `self.board` in `__init__`. Take care to revert it afterwards.

Things to know for later

- **Own-king capture bug (Task 3):** `capture_move` lets a plain `R` "capture" its own `RK`, because `"RK"` is not in `(".", "R")`. My legal-move check mirrors what the game really allows,

so it treats that as a legal move. Fixing it belongs in Task 3.

- **Kings currently only move forward** like ordinary pieces, because king movement is also Task 3. A `BK` on row 7 therefore has no legal moves today, so a position with only such pieces would be reported as "no legal moves". That will correct itself once kings can move in both directions.
- **No draws or repetition rules** are included, since the README doesn't ask for them.







board_fab648.py

game_8ac505.py

main_1.py

README_5a1b8b.md

rules_12ba2a.py

test_task1.py

test_task2.py

Tasks 1 and 2 have been applied locally. I ran `python test_task1.py` (7 tests passed) and `python test_task2.py` (12 tests passed). I also manually confirmed that a capture removes the jumped piece.

Now inspect the README and current source files, and address Task 3 — Complete Checkers rules — according to the README.

Check and implement any missing requirements for:

1. Mandatory captures: if a legal capture is available, a player cannot make an ordinary move.
2. Multi-capture: if another capture is available after a jump, the same piece must continue capturing before the turn ends.
3. Kings: promoted kings can move and capture diagonally in both directions.
4. Promotion: regular pieces become kings when they reach the opponent's back row.
5. Correct piece ownership: a regular piece must never capture its own piece, including its own king.

First identify which requirements already work and which are missing. Make only the changes needed for Task 3, preserving the Task 1 capture fix and Task 2 win/no-legal-moves detection. Keep the existing project modular, use in-memory state, and do not add dependencies or unrelated gameplay features. Do not implement Task 4's player-facing move feedback yet.

Provide the full contents of every changed file, explain the changes in simple terms, and supply reproducible tests for forced captures, multi-captures, king movement in both directions, promotion, and attempted captures of friendly pieces. If possible, run these tests and the existing

Task 1 and Task 2 regression suites; clearly state which tests were actually executed.

Implementing capture-chain enforcement and move validation logic.

Implementing capture-chain enforcement and move validation logic.

Done

I'll start by finding out which of the five requirements already work, by running the current code against each one.

Ran command

Baseline tests and probe which Task 3 requirements already work

Done

Baseline confirmed. Both earlier suites pass (7 and 12). Only promotion already works; the other four requirements are missing or broken. I'll now implement the missing ones.

Ran command

Implement Task 3 in rules.py and game.py, rerun regressions and probe

Done

All six probes now behave correctly and both regression suites still pass. Next I'll write the full Task 3 test suite.

Ran command

Write and run the Task 3 test suite

Done

Continue Task 3 from where you stopped. You confirmed that Tasks 1 and 2 pass their regression tests and that promotion already works.

Implement only the missing Checkers rules, preserving Tasks 1 and 2. Provide the updated files and a test_task3.py test suite. Run the Task 3 tests and both existing regression suites (7 Task 1 tests and 12 Task 2 tests). Keep the explanation concise and clearly report the test results.

Ran command, presented file

Run all three suites together and copy deliverables

Presenting file(s)...

Done

Task 3 is done: forced captures, multi-captures, two-way king movement and the friendly-piece capture fix are in, and all 54 tests pass. The changes are in `rules.py` and `game.py`; `board.py` is unchanged, and `test_task3.py` is the new suite.

What was already working and what was missing

I probed each requirement against your current code before changing anything.

Requirement	Before	Now
Promotion at the opponent's back row	Already worked	Unchanged; tests added
Mandatory captures	Missing (an ordinary move was accepted while a capture existed)	Fixed
Multi-capture	Missing (the turn passed after one jump)	Fixed
King movement and capture in both directions	Missing (kings were treated like ordinary pieces)	Fixed
Never capture own pieces	Broken (a man could jump its own king)	Fixed

What changed

`rules.py`

- `simple_move` and `capture_move` now check whether the piece at the start square is a king. A king may go both ways; an ordinary piece only forward.
- `capture_move` now only allows jumping an opponent's piece (man or king). That closes the own-king bug.

- Two new helpers: `has_capture` (can this piece capture?) and `player_has_capture` (can any of the player's pieces capture?).

`game.py`

- A new `self.chain` field remembers a piece that must keep jumping.
- An ordinary move is rejected with `Capture is mandatory.` when any capture is available. You may still choose which capture to make.
- After a capture, if the same piece can jump again, the turn stays with the player. Any other move is rejected with `You must keep capturing with the piece at r c.`
- Task 1 capture removal and Task 2 game-over detection are untouched. A chain that takes the last piece still ends the game.
- Task 2's known gap is also fixed: a king on its back row used to look stuck, and now it counts as having moves.

Rule choice to know about

A man that is crowned in the middle of a capture ends its turn there, even if the new king could keep jumping. This is the standard American checkers rule, and there is a test for it. The README doesn't specify it, so tell me if your course expects otherwise.

Test results (all actually executed)

Suite	Result
`test_task1.py`	7 tests, OK
`test_task2.py`	12 tests, OK
`test_task3.py`	35 tests, OK
All three together	54 tests, OK

The 35 Task 3 tests cover:

- forced captures (7)
- multi-captures, including a triple zigzag, blocked mid-chain moves, and a chain that ends the game (8)
- king moves and captures in both directions (10)
- promotion (4)
- friendly-piece captures (6)

Not done: I did not play interactively through `python main.py`. Each test sets up a position directly on the board and drives the game with scripted input, so for your "after" video you will need to set

up positions yourself.

Task 4's move/capture/promotion feedback is not implemented. The only new messages are the two rejection messages above.

rules
Py

game
Py

test_task3
Py

board_842850.py

game_9a4c12.py

main_d5afc6.py

README_5cfbb7.md

rules_5311af.py

test_task1_1.py

test_task2_1.py

test_task3.py

Tasks 1–3 have been applied locally. The regression suites pass: Task 1 (7 tests), Task 2 (12 tests), and Task 3 (35 tests).

Now implement Task 4 — Move-level feedback, following the requirements in the uploaded README.

Provide clear feedback for accepted ordinary moves, captures, and promotions. Capture messages should identify the captured piece and its coordinates; promotion messages should identify the new king. Avoid duplicate or contradictory messages, including during multi-capture turns.

Preserve all existing functionality and the modular design. Make only the necessary changes and provide the full contents of changed files plus a test_task4.py suite.

Run the new tests and all three existing regression suites. Clearly report the actual results. Do not add unrelated features, dependencies, or persistent storage. Keep the explanation concise.

Writing the rejection and game-over cases for move messages.

Writing the rejection and game-over cases for move messages.

Done

I'll set up from your uploaded files and confirm the baseline first. Then I'll implement the feedback and test it.

Ran command

Set up working dir, run baseline regression suites, show current (silent) output

Done

The baseline is 54 passing, and the game prints no move feedback at all. The edit needs one result line per accepted move, so I'll print once, after the move is applied.

Ran command

Implement Task 4 feedback in game.py and rerun regressions

Done

Only `game.py` changed, and the 54 existing tests still pass. Now the Task 4 suite.

Ran command

Write and run the Task 4 test suite

Done

All 19 pass. Next I'll run all four suites together, show a real play-through of the output, and package the files.

Ran command, presented file

Run all four suites, show real output, copy deliverables

Presenting file(s)...

Done

Task 4 is done: every accepted move now prints exactly one result line, and all 73 tests pass (7 + 12 + 35 + 19). The only source file that changed is `game.py`; `board.py` and `rules.py` are untouched, and the new suite is `test_task4.py`.

What changed in `game.py`

- A small helper, `_describe_move(...)`, builds one line per accepted move. It is called once, after the move is applied.
- Before the board changes, the code records which piece moved and what it jumped, so a capture can name the victim and its square.
- The message sits next to the existing promotion and multi-capture logic, so those rules are unchanged. Rejected moves keep their existing messages and print no result line.

Message formats

Situation	Output
Ordinary move	`R moves (5,4) -> (4,5).`
Capture	`R captures B at (3,4): (4,5) -> (2,3).` (a captured king shows as `BK`)
Promotion	`R moves (1,2) -> (0,1). Promoted to king: RK at (0,1).`
Promotion by capture	`R captures B at (1,4): (2,3) -> (0,5). Promoted to king: RK at (0,5).`
Mid-chain jump	`R captures B at (5,2): (6,1) -> (4,3). Jump again from (4,3).`

Promotion and "Jump again" are added to the same line instead of printed separately. That keeps one accepted move to one result, and a crowning jump never says "Jump again", because a newly crowned piece ends its turn (the rule chosen in Task 3). In a multi-capture turn, each jump gets its own line, and only the last has no "Jump again".

Test results (all executed)

Suite	Result
`test_task1.py`	7 tests, OK
`test_task2.py`	12 tests, OK
`test_task3.py`	35 tests, OK
`test_task4.py`	19 tests, OK
All four together	73 tests, OK

The 19 new tests cover:

- ordinary and king moves
- captures, including a captured king and a king capturing backward
- promotion by move and by capture, and a king not being "re-promoted"
- double and triple jumps with no duplicate lines
- rejected moves, bad input and quitting printing no result line
- ordering and uniqueness when a capture ends the game

I also ran a scripted play-through. It showed the three messages for your opening sequence, `5 4 4 5`, `2 3 3 4`, `4 5 2 3`, plus a double jump and a promotion by capture.

Not covered

I did not run `python main.py` interactively. The README's "after" video will need positions set up by hand, since a double jump or a promotion can't be reached in 10 seconds of normal play from the start board.

